

Analog-AI Hardware Accelerators for low-latency Transformer-based Language Models (Invited)

G. W. Burr¹, H. Tsai¹, I. Boybat², W. A. Simon², J. Büchel², A. Vasilopoulos², P. Narayanan¹, A. Fasoli¹, K. Hosokawa³, M. Le Gallo², M. Ishii³, Y. Kohda³, A. Okazaki³, A. Chen¹, C. Mackin¹, E. Ferro², K. El Maghraoui⁴, H. Benmeziane², T. Philicelli⁵, C. Lammie², A. M. Friz¹, J. Luquin¹, S. Jain⁴, A. Sebastian², and V. Narayanan⁴

¹ IBM Research–Almaden, San Jose, CA, USA, ² IBM Research–Zurich, Rüschlikon, Switzerland, ³ IBM Tokyo Research Laboratory, Tokyo, Japan, ⁴ IBM Research–T. J. Watson Research Center, Yorktown Heights, NY, USA, ⁵ IBM Albany NanoTech, Albany, NY, USA.

Analog Non-Volatile Memory-based accelerators offer high-throughput and energy-efficient Multiply-Accumulate operations for the large Fully-Connected layers that dominate Transformer-based Large Language Models (LLMs). We describe recent chip-demo and architectural efforts, quantify the unique benefits of Fully- (rather than Partially-) Weight-Stationary systems, and discuss factors affecting latency of token-processing and generation.

1 Introduction

Scaling the depth and width of Deep Neural Network (DNN) models has been a key driver of progress in artificial intelligence, first in image recognition [1] and more recently in Large Language Models (LLMs) [2, 3]. This trend has greatly increased the amount of compute, energy and raw data needed to train these very large networks with vast datasets [3, 4], as well as to perform rapid inference on the resulting models [3]. The Graphics Processing Units (GPUs) used for these compute tasks have scaled greatly in capability (and quantity), but only modestly in energy-efficiency. Reductions in computational precision since 2015 [5] have now mostly saturated, and the benefits of sparsity [6] are much less applicable to the large Fully-Connected (FC) layers found in LLMs – leaving technology node as the primary, but rapidly narrowing, path to still higher efficiency.

Recently, numerous researchers are exploring Compute-In-Memory (CIM) approaches [7, 8, 9] to increase energy-efficiency by performing Multiply-Accumulate (MAC) operations within ON-chip memory “Tiles,” to greatly reduce the motion of model-weights and partial sums. By exploiting such weight stationarity over short timespans with Volatile Memories (VMs) – SRAM or e-DRAM [7, 9] – or over longer timespans with slower, finite-endurance Non-Volatile Memories (NVMs) – Flash [10], Resistive-RAM [11], or phase-change memory (PCM) [12, 13] – such CIM approaches can offer high throughput, low latency, and high energy-efficiency [7, 8, 9]. Several NVM technologies support 3D layering of multiple device-arrays, or “tiers,” either by monolithic stacking in the Back-End-Of-the-Line [14], or with the vertical NAND-strings in 3D NAND-Flash [15].

2 Transformer-based networks

While these efficiency and speed benefits can be seen across all DNN models, they are particularly pronounced for workloads that exhibit large FC layers, such as **Transformer-based models** [2, 16, 3]. Transformer models, including BERT-base [16], BERT-large [16], and Llama-3.1-8b [17], use a stack of 12, 24, or 32 identically-structured layers, each comprising 4 large FC-layers (In-projection, Out-projection, FC1, FC2) and a self-attention block (**Fig. 1**). In Mixture-of-Expert (MoE) models [19, 20, 21], the FC1 and FC2 are made significantly larger but are organized into “experts,” allowing a router to conditionally select only the necessary experts (subset of FC1/FC2 weights) for any particular inference task. FC-compute operations (OPs) typically dominate the workload, but scale linearly with sequence-length S ; attention-compute OPs scale as S^2 .

Encoder-based models perform Natural Language Processing (NLP) classification-tasks (sentiment analysis, etc.) by pushing an entire sequence of S “tokens” through the layer-stack, using all S^2 elements of each attention-matrix (within each of the attention-heads,

within each layer, **Table 1**). After pre-filling a user-provided prompt, *decoder-based* models generate one or more new tokens with a similar pass. Here the attention-block uses masking (and potentially, sparsity) to reduce the amount of attention compute [3, 22].

In an encoder-stack, tokens need to be staged before each attention-compute block, requiring significant local scratchpad. In contrast, the *masked self-attention* in a decoder-stack requires only a few incoming tokens before the next outgoing tokens can be injected into the subsequent Out-Projection block. Such a decoder-stack is frequently used in an auto-regressive *generative* manner, to predict the next (few) most-likely token(s). Extensive re-compute can be avoided by preserving *KV-cache* [23] data from previously-generated tokens, although the size of this cache can exceed the size of weight matrices for long sequences. Note that encoder-based models are trained in two steps – extensive *pre-training* that typically only needs to be performed once, followed by *fine-tuning* for the particular classification task at hand. Ultra-large LLMs are so capable after pre-training that often only simple “prompt engineering” is required.

The sheer size of these Transformer-based LLMs calls for more efficient AI compute. While the *macro*-level energy-efficiency of a CIM-Tile is demonstrably high [7, 9], it is non-trivial to deliver high *system* energy-efficiency – but careful design can implement the necessary Auxiliary digital compute, and can communicate excitation-vectors between Tiles and Compute-cores with reasonable energy and time [18]. VM-based CIM systems also need to occasionally move weight-vectors, in order to reuse the same small set of Tiles for many different FC-layers comprising different weight matrices.

In a **Fully Weight Stationary** (FWS) system [24], unique Tile resources are dedicated for the execution of each dense-weight block within the LLM, allowing implementation of LLM workloads with absolutely no weight-motion. In a **Fully-unrolled** FWS (F-FWS) system, enough unique compute-core resources are provisioned for attention-compute blocks, staging, LayerNorm, and other vector operations that each and every internal excitation-vector is processed with minimal delay, with no excess time lost waiting for resources. While such system requires both a unique weight location (NVM CIM-device, VM CIM-device, or conventional scratchpad entry) for each and every trained weight in the entire system, and significant resources for each attention-block, an F-FWS system is the only configuration which supports full pipelining and a “continuous input” mode for maximum throughput and low latency. In such a system, it is not necessary to choose a batch-size at all. That said, the silicon area required to implement large models may not be feasible, necessitating some degree of **Partial Unrolling** and **Partial Weight Stationarity**, in which batching is used to help amortize ...

1. the reuse of digital Compute-cores across multiple layer-blocks [21], representing a *Partially-unrolled* FWS (P-FWS) system;
2. the reuse of NVM CIM-Tiles across multiple layer-blocks [24] (also P-FWS);
3. the shuffling of weight- and/or KV-cache data among digital scratchpads [25, 26] or SRAM CIM-Tiles [24, 9], representing a **Partially Weight Stationary** (PWS) system;
4. reloading of weight-data to scratchpads or SRAM CIM-Tiles from secondary SRAM or external DRAM [27, 9, 24] (also PWS);
5. reloading of KV-cache data from external DRAM [23] (an available choice for any system, to avoid imposing a ceiling on maximum model-size and sequence-length).

Note that we assume that any FWS system (either F-FWS or P-FWS) can and will use multiple 3D tiers on some reasonable subset of NVM CIM-Tiles to implement any given dense-weight block. Such a system remains FWS so long as those Tiles become immediately available to process the next token entering that same dense-weight block. In contrast, in a P-FWS system as described above (#2), excitation-vectors located at different stages within the model must inevitably wait for availability of the shared “One Tier at a Time [21]” NVM CIM-Tile resource, reducing throughput and increasing latency.

In this paper, we refine the work begun in Ref [24] to quantify the benefits of F-FWS for encoder-style Transformers, and lay the qualitative groundwork for extending this work to an F-FWS implementation of large decoder-style Transformers. This includes exploring when constraints #1, 2, and 5 listed above would force an NVM CIM-Tile system from a fastest-latency F-FWS configuration into a slower P-FWS implementation.

3 Chip demonstrations of various DNN

Compute-In-Memory (CIM) Tiles are the primary MAC-compute and weight storage units for the DNN inference workload. In NVM-based accelerators, the conductance G of the analog resistive device implements synaptic weight, while voltage level or duration implements input activations. Activations are converted into analog pulse durations, enabling parallel excitation of the analog memory array. Multiply-accumulate operations are performed by the combination of Ohm's law and Kirchhoff's Current law (**Fig. 4c**) [29, 8]. The resulting raw summations are converted from analog current and integrated charge back into digital vectors, which are then transmitted using the 2D Mesh. Conversion is performed in parallel using dedicated Current-Controlled Oscillator (CCO)-based Analog-to-Digital Converters (ADCs) [13].

Each NVM CIM-Tile must implement accurate MAC operations, despite the presence of device and circuit noise, circuit nonlinearities, and quantization effects. The resulting excitation vectors must then efficiently move between Tiles and Compute-cores used for efficient Auxiliary compute, leading to DNN results at high throughput, high energy-efficiency, and low-latency – delivered at or near the software-equivalent DNN accuracy exhibited by the original DNN model (as implemented on a CPU or GPU using 32-bit Floating-Point (FP32) compute). We describe several chip-demonstrations that extend the boundaries of total number of DNN weights implemented and complexity of excitation orchestration, the integration of highly-linear, well-characterized and optimized NVM CIM-Tile compute, and the implementation of Auxiliary compute at or near the CIM-Tiles.

Recently, a large Recurrent Neural Network Transducer (RNNT) model (**Fig. 2a**) was demonstrated [28], using 5 chips to encode 45M weights using >140M PCM devices. Tightly-orchestrated transport of massively-parallel excitation vectors was implemented using a 2-D mesh similar to **Fig. 4** (here conveying durations rather than digital-bits). While nearly all MAC operations were performed on-chip, the remaining digital operations are performed off-chip. Near software-equivalent accuracy (<2% degradation in Word Error Rate) was measured on the Librispeech dataset, along with an estimated $14\times$ benefit in system performance, expressed as TOPS/W or Samples/s/W, over existing MLPerf digital submissions based on GPUs. This chip-demo implemented a large number of DNN weights, together with experimental demonstration of the 2-D circuit-switched mesh concept.

On a second chip, we integrated CCO-based ADCs (one per integration-row) into PCM-based CIM-Tiles [30] (**Fig. 3**). This chip also features digital Auxiliary-compute for affine-scaling and complex activation-functions (ReLU, sigmoid, tanh) at or near the CIM-Tiles, and was used to demonstrate an LSTM-based image-captioning network with high accuracy (**Fig. 3**). All computation (fully-connected layers, LSTM vector-operations) was performed on-chip; excitation-vectors were pulled off-chip for data-motion between chip-blocks.

4 Hardware-Aware Training of DNNs

We can further increase the robustness of DNN models by performing hardware-aware (HWA) training, preparing the network for the various imperfections in the Matrix-Vector-Multiplications (MVMs) performed on NVM-based CIM-Tiles. These arise from a variety of noise sources, nonlinearities and quantization effects. HWA training requires a model that is accurate enough to be predictive while still being amenable to the rapid computation needed to perform DNN training in reasonable time. HWA is analogous to quantization-aware training (QAT) methods now commonly used when deploying DNNs onto reduced-precision digital hardware [31].

For instance, the image captioning network shown in **Fig. 3** was trained with HWA techniques using a noise-model carefully matched to measured characteristics of the NVM-CIM tiles on the 64-Tile chip. In fact, starting with a model trained to high-accuracy with FP32, HWA training can greatly enhance the robustness of a variety of deep neural networks (DNNs) including recurrent neural networks (RNNs), Transformers and Convolutional neural networks (CNNs) [32, 30]. In general, recurrent networks (such as LSTMs and RNNT) are the most robust to NVM-based CIM-Tile imperfections.

Transformer models are reasonably robust, although models with fewer degrees-of-freedom tend to be less robust. For instance, ALBERT has the same number of operations as BERT-base but reuses the the same weights for all layers. While this may decrease the model footprint, there is a cost in both raw test-error performance and in robustness. In contrast, using HWA training techniques to perform the fine-tuning for each GLUE classification-task can provide fully software-equivalent accuracy on BERT-base, at least until the accumulated effects of one year of PCM conductance-drift after programming [32]. Recently, we demonstrated that MoE-type LLMs scale favorably for NVM CIM-based systems, both in terms of latency as well as model accuracy for a given inference-OPs budget. Furthermore, HWA-trained MoEs can maintain this accuracy performance even in the presence of noise typical of analog computations [21].

As a result, we should be able to accurately program NVM CIM-Tiles with DNN model weights made highly robust with HWA-based fine-tuning, and have an expectation to obtain the same accuracies as any digital system using ~ 4 -bit weights and 8-bit excitations. Now we turn to a discussion of an architecture that integrates this compute of dense fully-connected weight-layers with the digital compute and vector-communication needed for high throughput, low latency and achieving a high *system* energy-efficiency given the large *macro* energy-efficiency of the NVM CIM-Tiles.

5 Analog-Fabric Architecture

We recently introduced a highly programmable Analog-AI and Compute-In-Memory accelerator architecture [18], which utilizes a highly specialized set of Compute-cores and Tiles arranged within a common building block known as an Analog Fabric (AF) (**Fig. 4**), together with SRAM scratchpads, IO blocks, and a 2D Mesh on top of the building blocks. Border-Guard circuits (BGs) are placed between components to control dataflow over the 2D mesh for high throughput and energy-efficiency across a wide variety of workloads, supporting multi-casting of data-vectors traveling from Compute-cores to Tiles. This on-chip interconnect helps enable system-level utilization and efficiency well beyond current AI accelerator systems. Multiple AFs can share the same chiplet, which can be tiled for system scale-up.

Given the finite endurance, and the slow and power-hungry programming of NVM-devices, such Analog-AI systems must be Fully-Weight-Stationary (FWS) – every weight must be pre-programmed before execution of the inference workload begins. While this requires unique NVM-devices for each and every weight in the model, there are numerous benefits. No time or energy is spent moving weights during the workload, allowing the maximum degree of pipelining supported by the model. Adverse tradeoffs between efficiency and latency are avoided, so that the system inherently reaches maximum throughput *and* energy-efficiency independent of batch-size. Excess padding of short sequences is also not required.

By optimizing a spatial distribution of Tiles and Compute-cores to meet the needs of diverse networks (including LSTMs, Transformers, and CNNs), together with a well-optimized 2D mesh-interconnect, we showed that this Analog-AI CIM-based architecture can be $40\times$ – $140\times$ more energy-efficient than the NVIDIA A100 (compared at $b=8$) across networks of interest [18]. The NVM-Tile assumed here is based on PCM, with characteristics (100 TOPS/W at 40 nsec integration-time) taken from 14nm-node circuit-simulations [18].

6 Benefits of Full Weight Stationarity

In Reference [24] we raised the question of whether the effort of providing all the NVM CIM-Tiles needed for a Fully-Weight-Stationarity

(FWS) system can be justified. After all, a Partially-Weight-Stationary (PWS) system could greatly reduce the number of Tiles, especially if weight-reloading can be infrequent enough. To answer these questions, we employed the same performance simulator used in Ref [18]. In this paper, we plot the results that were shown in the conference paper, corrected from the original submission [24]. This system is both *Fully Weight Stationary*, with unique 2D NVM CIM-Tiles for each trained-weight block, and *Fully-unrolled*, with unique Compute-Cores for intervening digital compute.

Since the AF used in Ref [18] was optimized for performance across a wide variety of workloads, we introduced two new AFs **Fig. 4(a,b)**, designed specifically for BERT-base and BERT-large, containing exactly enough tiles, MC-cores, CC_L and CC_H cores for one layer of BERT-large (a few Compute-cores and Tiles are power-gated for BERT-base) [24]. Die-to-Die (D2D) I/O cores were added to provide sufficient interconnect between F-FWS chiplets. Both systems use exactly the same Tile characteristics (100 TOPS/W, 40nsec integration). This Tile performance (10.9 TOPS/sq.mm) matches or exceeds existing SRAM-based CIM-Tiles [33].

Our AF architecture was adapted for VM-based CIM tiles, to implement a highly-efficient PWS system by adding additional scratchpad (MC) cores (**Fig. 4(b)**), both for staging of excitations ($b \times S$ tokens) each time the workload passes through the currently-programmed VM-CIM tiles, and for weight-storage across all the layers (up to 12 MB, 10 sq.mm per MC). (In an alternative not studied here, weights could be loaded from DRAM, reducing chip area albeit at a significant energy cost.) To reduce data-traffic and reduce weight rewriting, odd layers progress from south to north, even layers from north to south. These choices, and the fraction of the network implemented between reloading, will drive interesting tradeoffs across sequence-length and batch-size (here $S=128$ and $b=8$).

Fig. 5 shows the trajectory of the 8 sentences through the 12 layers of BERT-base, for both PWS and F-FWS systems, and **Table 2** shows the results. These results are produced by a detailed architectural simulator[18], which schedules the, for BERT-Large, up to 625.3 billion OPs ($24 \text{ layers} \times 8 \text{ sentences} (b=8) \times \text{sequence-length of } 128$) as 205.6 million “jobs,” in order to compute the energy costs and time-constraints of data-transport and compute, while respecting the data causality required by the DNN model.

Although the F-FWS system does occupy more area, the system throughput is significantly higher and latency is lower (up to $9.75 \times$ lower for BERT-Large). Thus TOPS/sq.mm is maintained while offering markedly reduced latency — a benefit that is difficult to recover by simply duplicating the PWS system out to iso-area even when batch-size is artificially constrained to the same value as the PWS system. Furthermore, by exploiting the fact that an F-FWS system does not need to restrict itself to any given batch-size, such a system more than pays for its larger area in “continuous input” mode, exhibiting an improvement of $5.21 \times$ in computational area efficiency (in [TOPS/seq/sq.mm.]) and $33.3 \times$ better throughput enhancement, as compared to a PWS system with exactly the same Tile characteristics. **Fig. 6** puts these quantitative performance results in context to each other and to prior SRAM- and NVM-based CIM studies, showing that the highly-pipelined nature of an F-FWS system does a better job — at turning the inherently high *macro* computational area efficiency of each CIM-Tile into good *system* performance — than a PWS system that is forced to reuse the same small number of resources across the model for each batch.

7 Factors affecting latency for scaled LLMs

It is clear that 2D NVM CIM-Tiles will be insufficient for LLM models with billions of weights. Our approach here is to move to 3D tiles [21], either using monolithically-stacked PCM or RRAM devices or 3D NAND-Flash technology, with each tile pre-programmed with as few as 4-8 to as many as several hundred “tiers.” As with a 2D NVM CIM-Tile, an input data-vector arriving at the west edge of a tile continues to perform a Matrix-Vector-Multiplication of size 512×512 (or perhaps 1024×1024). However, since this operation is performed on just one of the “tiers” associated with this Tile, the overall weight-

capacity of a NVM-based system is greatly increased without increasing silicon area. Furthermore, by performing affine-scale and digital aggregation operations at the edge of the Tile, the number of post-integration data-transport steps involving the 2D data-transport mesh can be significantly reduced [34].

Fig. 7 illustrates the progression of a single token through any Transformer layer, through four trained-weight layer-blocks (In-projection, Out-projection, FC1, FC2) interspersed with light digital-compute (two LNORM blocks, Gelu blocks), two staging resources for residual connections, and heavy digital-compute cores for implementing the Attention or “QKV” compute. While text annotations refer specifically to the model requirements of Llama-3.1-8b [17], including the additional “gate” projection layer that doubles the size of FC1, these same layer-blocks are found in all Transformers. In an F-FWS solution, the concept is that as each token moves up and to the right, the “next” token enters that same Tile or Core resource.

In an encoder or decoder undergoing prefill, the In-projection block is processing and storing tokens into SRAM provisioned within the QKV compute resource, even as other tokens are being pulled from elsewhere in the address-space of that same SRAM to feed QKV processing. This allows the processing of batches to be pipelined so that all resources are continuously busy. In a decoder in “generation” mode, the QKV block must rapidly process one (or a few) tokens arriving from the In-projection block, using stored KV-cache information to compute the next few rows of the masked attention matrix. The resulting output tokens are then sent to the Out-projection layer-block, and on through the remainder of the Transformer layer.

There are two main pain-points to implementing this Transformer layer at speed: the FC2 block and the QKV block. The former dominates when the sequence-length is short; the latter when the sequence-length becomes longer. At first glance, it might seem that the LNORM operations are also going to be difficult to execute in time — if each light Compute-core holds only 1024 (or 512) of the vector-elements, then 4 (or 8) of these cores must exchange their intermediate sum data in order to correctly compute the global mean and standard deviation across the 4,096 elements. However, while these LNORM operations are not trivial to implement, typically the summation of the FC2 block tends to dominate the rate at which tokens can be processed.

The FC2 block consists of a very tall matrix (14,366 elements in Llama-3.1-8b), requiring the summation to be broken into 14 portions of 1024 rows each (or 28 portions of 512 rows each). In a 2D NVM CIM-Tile system, this would require 14 (28) successive uses of the 2D mesh to transport the output vectors, although this number can be reduced by provisioning a short tree of additional light Compute-cores to perform partial sums. Moving to a 3D NVM CIM-Tile can help by allowing partial sums to be accumulated at the edge of each Tile. That said, if all of the summation were to be performed at the Tile-edge, then while the intermediate result-vectors are completely eliminated, the process would be forced to span 14 (28) uses of the 2D mesh to deliver all input vectors, or 14 (28) CIM-Tile integration periods, whichever is slower. A better approach is to deliver a subset of the 14 (28) input vectors to each CIM-Tile, and then complete the integration across all 14,366 elements by transporting a modest number of resulting partially-summed vectors to one (or a few) light Compute-cores to complete the aggregation process.

The task of the QKV block is centered on computing one or more rows of the attention matrix P . Each row is prepared by performing inner products between a single Q-vector and all relevant K-vectors. In an encoder, all K-vectors in the sequence are relevant; in a decoder with masked attention, only previous (e.g., lower index) K-vectors are relevant. After performing a softmax operation, a row of P then undergoes multiply-accumulate against all relevant V-vectors to synthesize a single output token. These operations are usually performed in parallel in a number of attention-heads, with their individual outputs concatenated to return from the attention-head size (128 in Llama-3.1-8b) to the hidden-size of the model (4,096). Furthermore, when multiple rows of the attention matrix are being computed in rapid suc-

session, the compute can be pipelined so that the QK preparation of the next row can be overlapped with the PV output of the previous row. In order to maximize throughput and minimize latency, it is highly desirable to parallelize the pass through the relevant K- and V-vectors, both across the sequence as well as across attention-heads, rather than implement these sequentially.

In implementing a decoder-based Transformer, the presence of a KV-cache avoids the need for extensive re-compute [23]. However, as sequence-lengths grow extremely long, the size of the KV-cache becomes a serious headache. This has led to the concept of “grouped query” attention (GQA), in which the same K- and V-vectors are used across several attention heads [35]. (Llama-3.1-8b uses 32 attention-heads but only 8 unique K- and V-vectors, per layer.) **Fig. 8** shows the advantages of using GQA, as well as the problems created by the use of batching (since each batch needs a different set of K- and V-vectors) and by a PWS system (since each layer assigned to the same QKV compute-resources also needs its own set of K- and V-vectors). Thus while the F-FWS system requires a large amount of silicon, it provides both significant latency and throughput advantages and minimizes the total amount of KV-cache memory by keeping batchsize small. In effect, each chip need only provide enough local memory to support vectors of two sentences ($b=2$) – the one being processed, and the subsequent sentence whose data is arriving in preparation for its processing. Furthermore, since this scratchpad memory is provisioned across multiple chips, the amount of SRAM needed per QKV compute-resource and per chip remains quite modest. In turn, this can greatly expand the model-size and sequence-length ceilings at which KV-cache data must be sent out to DRAM and recalled on demand. Additional batching, beyond $b=2$, can then be introduced to allow decoder pre-fill and generative tasks to be overlapped, as SRAM capacity at the QKV compute-core and/or retrieval from DRAM allows.

Finally, two important parts of an LLM sit outside the Transformer stack: the input-embedding table that prepares each token’s input-vector (using 1-hot lookup from a dictionary), and the output-embedding table that scores each output token’s output-vector against the entire dictionary. While the input-embedding table could potentially be implemented using a conventional address-based memory, the output-embedding table is effectively a dense weight-layer, making it attractive for implementation using NVM CIM-Tiles. However, this table is exceedingly large (for instance, the dictionary size for Llama-3.1-8b is 128,256), posing significant but achievable challenges for both accurate computation of this block, and for the implementation of argmax (potentially preceded by softmax) operations across such large vectors.

8 Conclusion

Analog non-volatile memory (NVM)-based accelerators can offer high-throughput and energy-efficient multiply-accumulate (MAC) operations for the large Fully-Connected (FC) layers that dominate Transformer-based Large Language Models (LLMs). We describe a highly heterogeneous and programmable accelerator architecture for DNN inference that combines Compute-In-Memory (CIM) Tiles for weight-stationary, energy-efficient MAC operations, together with heterogeneous special-function Compute-cores for auxiliary digital computation. Massively parallel vectors of neuron-activation data are exchanged over short distances using a dense and efficient circuit-switched 2D mesh, with analog compute implemented using Ohm’s law and Kirchhoff’s current law on arrays of resistive memory devices. If weights are accurately programmed onto NVM devices and MAC operations are sufficiently linear, competitive DNN accuracies can be obtained, aided by Hardware-Aware training techniques. Even though a Fully-unrolled Fully-Weight-Stationary (F-FWS) system does occupy more area, it offers so much throughput that it effectively pays for itself in terms of TOPS/sq.mm. We show that an F-FWS system can offer $33\times$ higher raw-throughput and nearly $10\times$ faster latency on encoder-based Transformer models, and can greatly simplify KV-cache requirements for decoder-based Transformer models.

References:

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Adv. Neur. Inf. Proc. Sys.* 25 (2012).
- [2] A. Vaswani et al, “Attention is All you Need,” *Adv. Neur. Inf. Proc. Sys.*, 2017, pp. 5998–6008.
- [3] T. B. Brown et al, “Language Models are Few-Shot Learners,” *arXiv:2005.14165*, (2020).
- [4] D. Amodei and D. Hernandez, “AI and Compute,” <https://openai.com/blog/ai-and-compute>
- [5] S. Gupta et al, “Deep Learning with Limited Numerical Precision,” *ICML*, June 2015, pp. 1737–1746.
- [6] T. Hoeffer et al, “Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks,” *J. Mach. Learn. Res.*, 22.1 (2021), pp. 10882–11005.
- [7] N. Verma et al, “In-Memory Computing: Advances and Prospects,” *IEEE Solid State Circuits Magazine*, 11.3 (2019), pp. 43–55.
- [8] G. W. Burr et al, “Ohm’s Law + Kirchhoff’s Current Law = Better AI: Neural-Network Processing Done in Memory with Analog Circuits will Save Energy,” *IEEE Spectrum*, 58.12 (2021), pp. 44–49.
- [9] N. R. Shanbhag and S. K. Roy, “Benchmarking In-Memory Computing Architectures,” *IEEE Open J. Sol.-State Circ. Soc.*, 2 (2022), pp. 288–300.
- [10] L. Fick et al, “Analog matrix processor for edge AI real-time video analytics,” *ISSCC*, Vol. 65. 2022, pp. 260–262.
- [11] J.-H. Yoon et al, “A 40nm 100Kb 118.44 TOPS/W Ternary-weight Compute-in-Memory RRAM Macro with Voltage-sensing Read and Write Verification for reliable multi-bit RRAM operation,” *CICC*, 2021, pp. 1–2.
- [12] P. Narayanan et al, “Fully On-Chip MAC at 14 nm Enabled by Accurate Row-Wise Programming of PCM-Based Weights and Parallel Vector-Transport in Duration-Format,” *IEEE Trans. Electr. Dev.*, 68.12 (2021), pp. 6629–6636.
- [13] R. Khaddam-Aljameh et al, “HERMES-core—A 1.59-TOPS/mm² PCM on 14-nm CMOS in-memory compute core using 300-ps/LSB linearized CCO-based ADCs,” *IEEE J. Solid State Circ.*, 57.4 (2022), pp. 1027–1038.
- [14] F. Pellizzer and A. Redaelli, “3DXpoint fundamentals,” *Semiconductor Memories and Systems*, 2022, pp. 253–276.
- [15] B. Kim et al, “A High-Performance 1Tb 3b/Cell 3D-NAND Flash with a 194MB/s Write Throughput on over 300 Layers,” *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, p. 28.2.
- [16] J. Devlin et al, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv:1810.04805*, (2018).
- [17] huggingface.co/meta-llama/Llama-3.1-8B.
- [18] S. Jain et al, “A Heterogeneous and Programmable Compute-In-Memory Accelerator Architecture for Analog-AI using Dense 2D Mesh,” *IEEE Trans. VLSI Systems*, 31.1 (2023), pp. 114–127.
- [19] N. Shazeer et al, “Outrageously large neural networks: The sparsely-gated mixture of-experts layer,” *arXiv:1701.06538*, (2017).
- [20] A. Q. Jiang et al, “Mixtral of experts,” *arXiv:2401.04088*, (2024).
- [21] J. Büchel et al, “Efficient Scaling of Large Language Models with Mixture of Experts and 3D Analog In-Memory Computing,” *Nature Computational Science*, (2025). in press.
- [22] R. Child et al, “Generating long sequences with sparse transformers,” *arXiv:1904.10509*, (2019).
- [23] R. Pope et al, “Efficiently scaling transformer inference,” *Proceedings of Machine Learning and Systems*, 5 (2023), pp. 606–624.
- [24] G. W. Burr et al, “Design of Analog-AI Hardware Accelerators for Transformer-based Language Models (Invited),” *IEDM 2023*, 2023.
- [25] D. S. Modha et al, “Neural inference at the frontier of energy, space, and time,” *Science*, 382.6668 (2023), pp. 329–335.
- [26] R. Appuswamy et al, “Breakthrough low-latency, high-energy-efficiency LLM inference performance using NorthPole,” *IEEE Conference on High Performance Extreme Computing (HPEC)*, 2024.
- [27] G. W. Burr et al, “Fair and comprehensive benchmarking of machine learning processing chips,” *IEEE Design & Test*, (2021), pp. 1–1.
- [28] S. Ambrogio et al, “An Analog-AI Chip for Energy-Efficient Speech Recognition and Transcription,” *Nature*, 620.7975 (2023), pp. 768–775.
- [29] P. Narayanan et al, “Toward on-chip acceleration of the backpropagation algorithm using nonvolatile memory,” *IBM J. Res. Dev.*, 61.4/5 (2017), 11:1–11:11.
- [30] M. Le Gallo et al, “A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference,” *Nature Electronics*, 6.9 (2023), pp. 680–693.
- [31] A. Agrawal et al, “9.1 A 7nm 4-Core AI Chip with 25.6TFLOPS Hybrid FP8 Training, 102.4TOPS INT4 Inference and Workload-Aware Throttling,” *ISSCC*, Vol. 64. 2021, pp. 144–146.
- [32] M. J. Rasch et al, “Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators,” *Nature Communications*, 14.1 (2023), p. 5282.
- [33] H. Jia et al, “Scalable and programmable neural network inference accelerator based on in-memory computing,” *IEEE J. Solid-State Circ.*, 57.1 (2021), pp. 198–211.
- [34] I. Boybat et al, “Heterogeneous Embedded Neural Processing Units utilizing PCM-based Analog In-Memory Computing,” *IEDM 2024*, 2024.
- [35] D. S. Modha et al, “GQA Training Generalized Efficient Transformers models from multi-head checkpoints,” *arXiv:2305.13245* (2023).

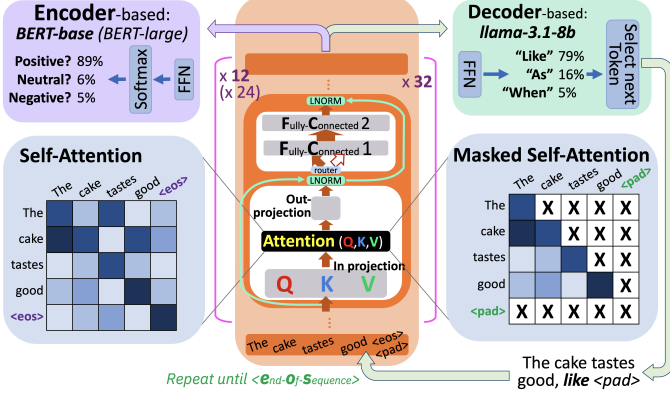


Fig. 1. Transformer-based models push sequences of tokens (vectors encoding words) through a stack of layers, each layer comprising 4 large FC-blocks and a self-attention (SA) block. Decoder-based architectures use a masked SA-block.

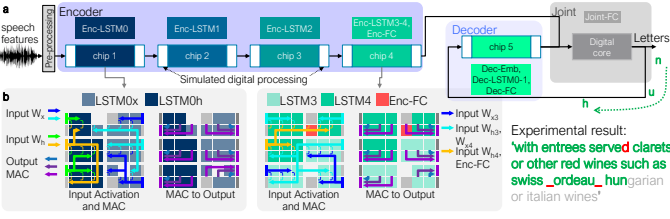


Fig. 2. (a) RNNT model transcribes input speech into words, character by character, using compute blocks mapped onto 5 chips (plus off-chip digital-compute). 45M weights are implemented with ~ 140 M PCM devices, (b) with complex routing and multicasting of input- and MAC-vectors across 34 Tiles. Example sentence from experimental results, at 9.258% Word-Error-Rate [28].

	BERT-base [16]	BERT-large [16]	Llama-3.1-8b [17]
type	encoder	encoder	decoder
# of layers	12	24	32
hidden vector-size	768	1,024	4,096
# of attn. heads/layer	12	16	32
KV-vector sharing	—	—	4
head-size	64	64	128
sequence-length	128 – 512	128 – 512	8,192
FC : attn. compute (at SL)	97% : 3%	98% : 2%	78% : 22%
# of weights	85M	302M	8B
CIM tier-size assumed	512×512	512×512	1,024×1,024
CIM tiers/layer	38	48	208
CC _H core (see Fig. 4)	v1 [18]	v1 [18]	v2
heads/CC _H core	4	4	2
CC _H cores/layer	3	4	16

Table 1. Transformer-based DNN models

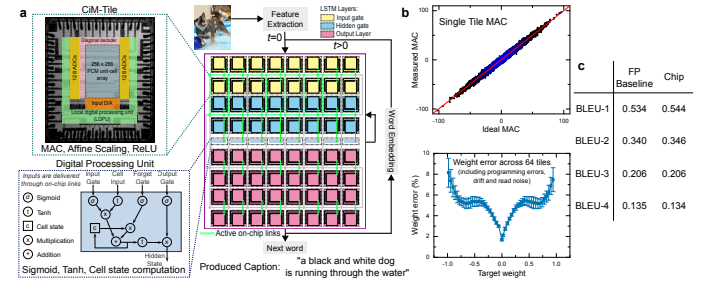


Fig. 3. a) A 64-Tile chip combines CCO-based ADCs and local digital-compute (scaling, ReLU activations) at each Tile, together with more complex digital processing (sigmoid & tanh activations, LSTM-vector OPs) at chip-center, to implement an image-captioning LSTM model. b) Highly-linear MAC operations and accurate vector-programming resulted in c) software-equivalent BLEU scores.

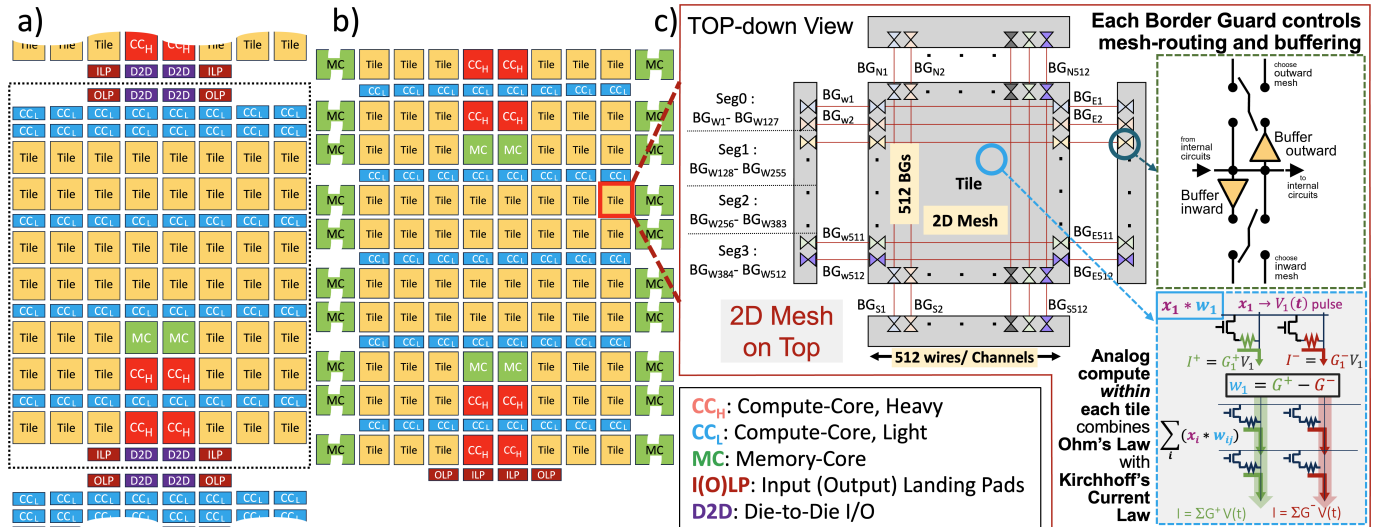


Fig. 4. A base chiplet for implementing BERT-base and BERT-large encoder-style Transformers is either a) repeated vertically to create a Fully-unrolled Fully-Weight-Stationary (F-FWS) system, or b) reused by weight-reloading to create a Partially-Weight-Stationary (PWS) system. Blocks such as analog-memory Tiles, Heavy and Light Compute-cores (CC_H and CC_L), scratchpads (MC), and both off-chip and Die-to-Die (D2D) IO are interconnected using c) a 2D Mesh with BorderGuard circuits (BGs) for dataflow control. In this F-FWS system, both Ohm's law and Kirchhoff's law allow Analog-AI compute using Non-Volatile Memory (NVM)-based resistors; in the PWS system studied here, weight-reloading mandates high-endurance SRAM (or e-DRAM), using Kirchhoff's law or charge sharing for MAC computation with multiple word- and/or bit-lines (not shown) [9], and large Memory-Cores (or off-chip DRAM, not shown) to store layer-weights. (Part (c) adapted from [18].)

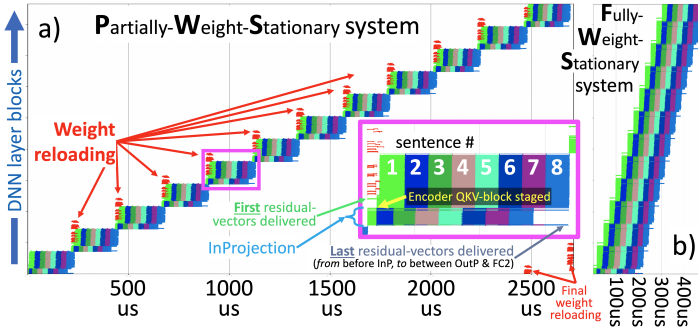


Fig. 5. Progress through BERT-base DNN-layer-blocks vs. time for the (a) PWS and (b) F-FWS systems. The timing of the 12-layers and $b=8$ sentences, the staging of all sentence-tokens before initiating self-attention (QKV) compute, and the staging of residually-connected data are all visible. PWS runs include weight-reloading before a CIM-Tile can be reused with different weights, and after the workload (to return the system to its initial state). From Ref [24] presentation.

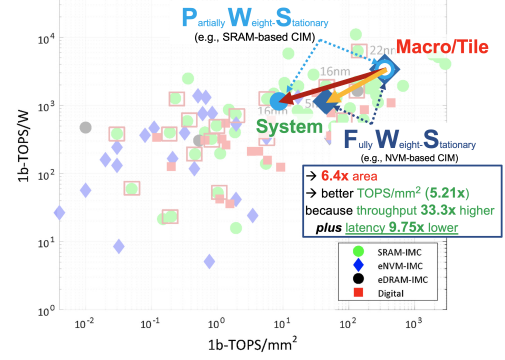


Fig. 6. 1-bit TOPS/W vs. 1-bit TOPS/sq.mm examples [9], with new data points from this study marked for Partially-Weight-Stationary (cyan solid circle) and Fully-Weight-Stationary (blue solid diamond) solutions capable of implementing BERT-large. Identical CIM-Tile performance (100 TOPS/W, 40nsec raw Tile-compute, open symbols) are assumed for PWS (e.g., SRAM-based CIM) and F-FWS (e.g., PCM-based CIM [18]), assuming 8-bit excitations and 4-bit (equivalent) weights. From Ref [24] presentation.

12-layer BERT-base	Partially WS	Fully WS	(continuous input) F-FWS is...
Area [sq.mm]	210	12×94	5.3× larger, but...
TOP/sec	65.2	359.8 (957.5)	5.52× (14.7×) faster
TOP/sec/W	26.4	28.3	1.07× as energy-efficient
TOP/sec/sq.mm	0.31	0.32 (0.85)	1.04× (2.76×) as area-efficient
latency [ms]	2.68	0.35	7.74× faster
24-layer BERT-large	Partially WS	Fully WS	F-FWS is...
Area [sq.mm]	350	24×94	6.4× larger, but...
TOP/sec	94.1	753.6 (3,135)	8.01× (33.3×) faster
TOP/sec/W	32.0	34.5	1.08× as energy-efficient
TOP/sec/sq.mm	0.27	0.34 (1.40)	1.2× (5.21×) as area-efficient
latency [ms]	6.62	0.68	9.75× faster

Table 2. Performance modeling results: Fully- vs. Partially-Weight-Stationary, for $b=8$ and S-128, from Ref [24] presentation.

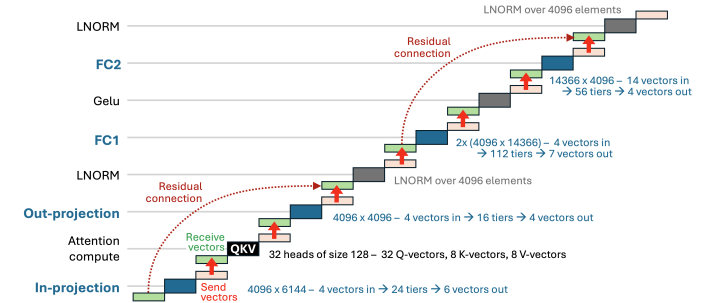


Fig. 7. Qualitative illustration of tokens moving through a single Transformer layer. Horizontal axis represents time (not to scale); vertical axis represents Transformer block. Each resource receives some number of input vectors, processes them either through Tiles or Compute-cores, and sends output vectors to the next resource. Annotations show requirements of the Llama-3.1-8b model [17]. Memory-cores (not shown) are needed to stage vectors for residual connections; significant SRAM memory is provisioned at/near the QKV Compute-core to store Q, K, and V vectors.

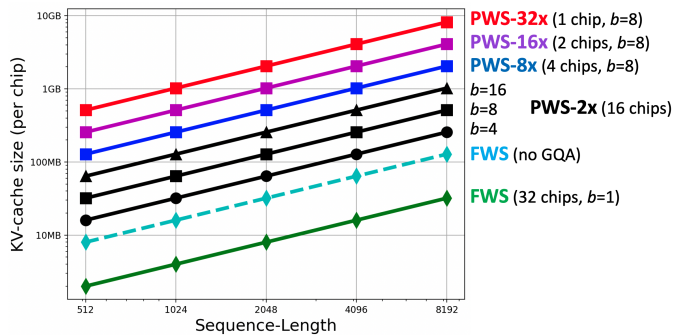


Fig. 8. KV-cache demands per chip, shown for Llama-3.1-8b [17] with 8-bit quantization of K and V vectors, increase significantly with sequence-length. While grouped query attention (GQA) helps reduce cache size, the choice of overlapping multiple layers onto the same resource (PWS-2x, 8x, etc.) and the resulting need to organize the compute into batches ($b=2, 4$, etc.) greatly exacerbate the amount of memory needed for KV-cache in each chip. While an Fully-unrolled Fully Weight Stationary (F-FWS) system requires a lot of silicon area for weights and uniquely-assigned Compute-cores, such a system not only pays for itself in throughput and latency (Table 2) but also helps keep KV-cache requirements per chip modest.

Acknowledgements:

We thank V. Mukherjee, W. Wilcke, S. Narayan, S. Munetoh, S. Yamamichi, M. Furmanek, C. Osborn, J. Burns, R. Divakaruni, M. Khare and the IBM Research AI Hardware Center for management and logistical support.